# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

| | | | |
|---|---|---|---|
| **Course Code:** | CSA10 01 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO2 | **Assessment:** | Assessment Tool 2 – Scenario-Based Requirement Analysis |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO2 Statement:** | *Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills* | | |
| **Student Name:** | Koppala Navya | **Reg. No.:** | 192511231 |
| **Date:** | 26/10/2026 | **Duration:** | 60 minutes |

### Code of Conduct

*IKoppala Navya(192511231) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Koppala Navya

## Annexure A – Scenario-Based Requirement Analysis

**Instructions to Students:**
Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

## Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

### Rubrics for Calculation

| Criteria | Excellent (4 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| **Scenario Understanding and Problem Analysis (30%)** | Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis. | Demonstrates good understanding with minor omissions. | Basic understanding with limited analysis. | Poor understanding; major aspects missing or incorrect. |
| **Functional and Non-Functional Requirement Identification (25%)** | Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements. | Identifies most requirements with minor classification errors. | Identifies only basic requirements; several omissions. | Requirements are incomplete, inaccurate, or poorly classified. |
| **Actor Identification and Use Case Modelling (20%)** | Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards. | Most actors and use cases identified; diagram has minor errors. | Limited actors/use cases identified; diagram partially correct. | Actors/use cases missing or incorrect; diagram absent or inaccurate. |
| **Software Requirement Specification (SRS)** | SRS is well-structured, complete, professionally organized, and | SRS is organized with minor missing sections or formatting issues. | SRS includes only essential sections with limited detail. | SRS is incomplete, poorly organized, or |

| | | | | |
|---|---|---|---|---|
| **Documentation (15%)** | includes all required sections. | | | lacks essential components. |
| **Assumptions, Constraints, and Requirement Validation (10%)** | Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification. | Identifies most assumptions and constraints; validation is adequate. | Limited assumptions/constraints; validation is superficial. | Assumptions, constraints, and validation are missing or incorrect. |

| Criteria | **Mark** |
|---|---|
| Scenario Understanding and Problem Analysis (30%) | /5 |
| Functional and Non-Functional Requirement Identification (25%) | /5 |
| Actor Identification and Use Case Modelling (20%) | /5 |
| Software Requirement Specification (SRS) Documentation (15%) | /5 |
| Assumptions, Constraints, and Requirement Validation (10%) | /5 |
| **TOTAL** | /25 |

## Recommend safe and efficient routes based on real-time and historical conditions.

**1.Study and Analyze the Scenario**
**1.1 Carefully understand the given problem statement.**

**Problem Statement**
Land acquisition projects often face delays due to problems such as incomplete documentation, legal disputes, compensation issues, approval delays, landowner objections, and poor coordination among departments. These delays can increase project costs and affect infrastructure development.
The proposed Predictive Analytics System for Early Detection of Land Acquisition Delays uses historical and current land acquisition data to identify projects that are likely to be delayed. The system analyzes factors such as approval status, documentation, compensation, legal cases, and acquisition progress and provides an early warning to officials.

**1.2Identify the business domain, stakeholders, existing challenges, and system objectives.**
Business Domain
- Land acquisition and infrastructure development
- Government administration
- Project management
- Predictive analytics and decision support

Stakeholders
- Land Acquisition Officers
- Government officials
- Project Managers
- Revenue Department officials
- Legal officers
- Landowners
- Data analysts
- System administrators

Existing Challenges
- Manual monitoring of acquisition progress
- Delayed identification of risks
- Incomplete or inconsistent records
- Legal and documentation issues
- Difficulty in predicting delays
- Lack of centralized monitoring

System Objectives
- Predict possible land acquisition delays at an early stage.
- Identify major factors causing delays.
- Provide risk levels for each acquisition case.
- Generate alerts for high-risk cases.
- Support officials in taking preventive actions.
- Improve project monitoring and decision-making.

**2. Identify Functional and Non-Functional Requirements**
**2.1 List all functional requirements required by the proposed system.**

1. User Login and Authentication
    - The system shall allow authorized users to log in securely.
    - Different users shall have different access permissions.
2. Land Acquisition Data Management
    - Add, update, delete, and view land acquisition records.
    - Store details such as land location, owner information, acquisition status, compensation, and approvals.
3. Data Validation
    - Check whether entered data is complete and valid.
    - Identify missing or inconsistent information.
4. Historical Data Management
    - Store previous land acquisition cases and their outcomes.
    - Use historical data for predictive analysis.
5. Predictive Analysis
    - Analyze acquisition data using predictive analytics or machine learning.
    - Predict whether a case is likely to be delayed.
6. Risk Classification
    - Classify cases into risk levels such as:
        - Low Risk
        - Medium Risk
        - High Risk
7. Delay Factor Identification
    - Identify factors contributing to possible delays, such as legal disputes, pending approvals, documentation issues, or compensation delays.
8. Alerts and Notifications
    - Generate alerts for high-risk acquisition cases.
    - Notify responsible officials about potential delays.
9. Dashboard
    - Display acquisition progress and risk information.
    - Show total cases, completed cases, delayed cases, and high-risk cases.
10. Reports

- Generate reports on acquisition progress and predicted delays.
- Allow reports to be viewed or downloaded.

11. Search and Filtering

- Search cases based on location, project, status, risk level, or acquisition ID.

12. Data Updating

- Allow authorized officials to update acquisition progress and status.

13. Audit Trail

- Record important user activities and changes made to the system.

**2.2 Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.**

Performance

- The system should provide predictions and dashboard results quickly.
- It should support multiple users simultaneously.

Security

- Only authorized users should access the system.
- Sensitive landowner and project information must be protected.

- Passwords and confidential data should be securely stored.

Reliability
- The system should provide consistent and accurate results.
- Data should not be lost during system failures.

Usability
- The interface should be simple and easy to understand.
- Officials should be able to access important information with minimal training.

Scalability
- The system should support increasing numbers of land acquisition projects and users.

Maintainability
- The system should be modular so that prediction models and features can be updated easily.

Availability
- The system should be available during working hours and preferably provide high availability.

Accuracy
- The prediction model should provide reliable delay predictions.
- The system should regularly evaluate and improve model performance.

**3. Identify Actors and Use Cases**

**3.1 Identify all primary and secondary actors interacting with the system.**

Land Acquisition Officer
- Login
- Add/update land records
- Monitor acquisition progress
- View predictions
- View risk levels
- Receive alerts
- Generate reports

Project Manager
- View project progress
- View predicted delays
- Monitor high-risk cases
- Generate project reports

Revenue/Department Officer
- Update land and approval information
- Verify acquisition records
- View acquisition status

Legal Officer
- Update legal dispute information
- View cases involving legal issues
- Monitor legal-related risks

System Administrator
- Manage users
- Manage roles and permissions
- Maintain system
- Monitor system activity

**3.2 List the major use cases associated with each actor.**

Landowner

- Provide required land information
- View relevant acquisition/compensation status where permitted

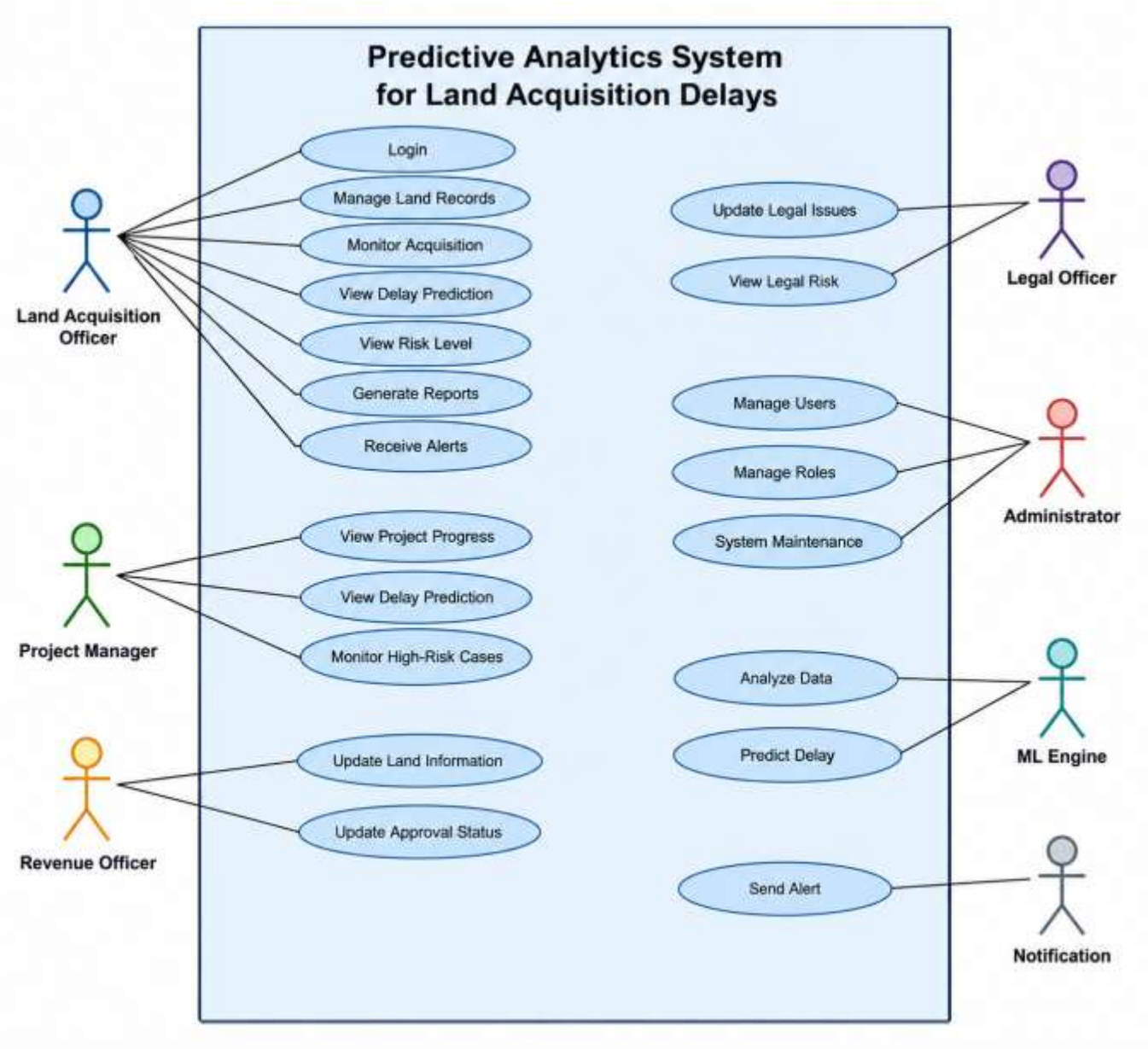
Predictive Analytics/ML Engine

- Process acquisition data
- Predict delay probability
- Generate risk classification

Notification Service

- Send alerts and notifications to responsible users.

**3.3 Draw a Use Case Diagram representing the interactions between actors and the system.**




**4. Prepare the Software Requirement Specification (SRS)**

**4.1 Introduction**

The Predictive Analytics System for Early Detection of Land Acquisition Delays is a software system designed to monitor land acquisition activities and predict potential delays. It uses historical and current acquisition data to identify risk factors and provide early warnings to responsible officials.
The system aims to improve land acquisition management, reduce project delays, and support data-driven decision-making.

### 4.2 Problem Description

Land acquisition is an important stage in infrastructure and development projects. However, acquisition activities may be delayed due to legal disputes, incomplete documentation, compensation problems, approval delays, objections from landowners, and administrative issues.
Traditional monitoring methods mainly depend on manual tracking, which may identify problems only after significant delays have occurred.
The proposed system addresses this problem by using predictive analytics to identify cases that are likely to experience delays before the delay becomes critical.

### 4.3 Scope of the System

The system will:

- Maintain land acquisition records.
- Track acquisition progress.
- Store historical acquisition data.
- Analyze important delay-related factors.
- Predict possible acquisition delays.
- Classify cases according to risk.
- Generate alerts for high-risk cases.
- Provide dashboards and reports.
- Help officials take preventive actions.

The system will primarily support monitoring and decision-making and will not automatically make legal or administrative decisions.

### 4.4 Functional Requirements

| ID | Requirement |
|---|---|
| FR1 | User authentication and authorization |
| FR2 | Add and manage land acquisition records |
| FR3 | Validate entered data |
| FR4 | Store historical acquisition data |
| FR5 | Track acquisition progress |
| FR6 | Analyze acquisition data |
| FR7 | Predict potential delays |
| FR8 | Classify cases into risk levels |
| FR9 | Identify major delay factors |
| FR10 | Generate alerts |
| FR11 | Display monitoring dashboard |
| FR12 | Generate reports |
| FR13 | Search and filter records |
| FR14 | Maintain audit logs |

### 4.5 Non-Functional Requirements

| Category | Requirement |
|---|---|

| | |
|---|---|
| Performance | Results should be displayed quickly |
| Security | Only authorized users can access data |
| Reliability | System should operate consistently |
| Usability | Interface should be simple and user-friendly |
| Scalability | System should support increasing data and users |
| Maintainability | System should be easy to update and maintain |
| Availability | System should be available when required |
| Accuracy | Predictions should be sufficiently accurate |
| Privacy | Personal and land-related information must be protected |

**4.6 Use Case Descriptions**

Use Case 1: User Login
Actor: All authorized users
Description:
The user enters valid login credentials to access the system.
Precondition:
The user must have a registered account.
Main Flow:

1. User enters username and password.
2. System validates credentials.
3. System verifies user role.
4. System provides access to the appropriate dashboard.

Postcondition:
User successfully enters the system.

Use Case 2: Manage Land Records
Actor: Land Acquisition Officer
Description:
The officer adds or updates land acquisition information.
Main Flow:

1. Officer logs in.
2. Selects land acquisition record.
3. Enters or modifies information.
4. System validates the data.
5. System stores the updated information.

Postcondition:
Land acquisition information is updated.

Use Case 3: Predict Acquisition Delay
Actor: Land Acquisition Officer / Project Manager
Description:
The system analyzes acquisition data and predicts the possibility of delay.
Main Flow:

1. User selects a project or acquisition case.
2. System collects relevant data.
3. Predictive model analyzes the data.
4. System calculates delay probability.
5. System displays risk level.
6. System identifies important contributing factors.

Postcondition:
Delay prediction is available to the authorized user.

Use Case 4: Generate Alert
Actor: System / Notification Service
Description:
The system generates an alert when a case has a high probability of delay.
Main Flow:

1. Prediction model identifies a high-risk case.
2. System creates an alert.
3. Notification is sent to the responsible official.
4. Alert is displayed on the dashboard.

Use Case 5: Generate Report
Actor: Project Manager / Land Acquisition Officer
Description:
Users generate reports related to acquisition progress and predicted delays.
Main Flow:

1. User selects report type.
2. System retrieves required information.
3. System generates the report.
4. User views or downloads the report.

**4.7 Data Requirements**
The system should maintain the following data:
Land Details

- Land ID
- Survey number
- Location
- Area
- Land type

Landowner Details

- Owner ID
- Owner name
- Contact details
- Ownership information

Acquisition Details

- Acquisition ID
- Project ID
- Acquisition start date
- Current status
- Expected completion date
- Actual completion date

Compensation Details

- Compensation amount
- Payment status
- Payment date

Legal Details

- Legal case status

- Dispute information
- Court case details

Approval Details

- Required approvals
- Completed approvals
- Pending approvals

Prediction Data

- Delay probability
- Risk level
- Predicted delay duration
- Major delay factors
- Prediction date

**4.8 External Interface Requirements**

User Interface

The system should provide:

- Login page
- Dashboard
- Land acquisition data entry forms
- Prediction screen
- Risk visualization
- Alert section
- Report generation page

Hardware Interface

The system can operate on standard computers, laptops, tablets, or servers with internet/network connectivity.

Software Interface

The system may interact with:

- Database management system
- Machine learning/predictive analytics module
- Authentication service
- Notification service
- Reporting system

Communication Interface

The system should support secure network communication using HTTPS.

**4.9 System Features**

1. Centralized Data Management

Stores land acquisition information in a centralized database.

2. Predictive Analytics

Analyzes historical and current information to predict possible delays.

3. Risk Dashboard

Displays cases according to their risk levels.

4. Early Warning System

Provides alerts for cases that require immediate attention.

5. Delay Factor Analysis

Identifies major factors contributing to predicted delays.

6. Reporting

Provides project-wise and department-wise reports.

7. Role-Based Access
Provides different access permissions to different users.

## 5. Identify Assumptions and Constraints

### 5.1 List the assumptions considered while developing the system.

1. Historical land acquisition data is available.
2. Users provide accurate and updated information.
3. Authorized officials have access to the system.
4. Required government approvals and records can be digitally maintained.
5. The predictive model can be trained using sufficient historical data.
6. Users have basic computer knowledge.
7. The organization has required network and computing facilities.
8. Prediction results are used as decision-support information and not as automatic legal decisions.

### 5.2 Identify technical, operational, economic, legal, and time-related constraints affecting the project.

Technical Constraints
- Quality of prediction depends on available historical data.
- Integration with existing government databases may be difficult.
- Internet or network failure may affect system availability.
- Machine learning models require regular maintenance.

Operational Constraints
- Officials need training to use the system.
- Data must be updated regularly.
- Different departments may use different data formats.

Economic Constraints
- Development and maintenance require financial resources.
- Cloud hosting, database, security, and infrastructure may increase costs.

Legal Constraints
- Landowner information must be protected.
- The system must comply with applicable data protection and government regulations.
- Legal decisions cannot be made solely based on system predictions.

Time Constraints
- Data collection and cleaning may require significant time.
- Training and testing the prediction model may take time.
- System deployment must be completed within the project schedule.

## 6. Validate Requirement Completeness and Consistency

### 6.1 Verify Whether All Stakeholder Requirements Have Been Addressed

The requirements of all major stakeholders have been considered in the proposed system.

| Stakeholder | Requirements Addressed |
|---|---|
| Land Acquisition Officer | Manage land records, monitor acquisition, view predictions, risk levels, alerts, and reports |
| Project Manager | Monitor project progress, view delay predictions, and identify high-risk cases |
| Revenue Officer | Update land information and approval status |
| Legal Officer | Update legal issues and monitor legal risks |
| Administrator | Manage users, roles, security, and system maintenance |

| | |
|---|---|
| ML Engine | Analyze acquisition data and predict possible delays |
| Notification Service | Generate and send alerts for high-risk cases |
| Landowner | Provide required land information and view permitted acquisition/compensation status |

Conclusion: The major stakeholder requirements are addressed through data management, monitoring, prediction, risk analysis, alerts, reporting, and administrative functions.

**6.2 Check for Ambiguity, Redundancy, Inconsistency, and Missing Requirements**

A. Ambiguity

Some requirements may not be sufficiently clear.

Examples:

- "The system should provide accurate predictions."
- "The system should respond quickly."
- "High-risk cases should generate alerts."

These requirements should define measurable values.

Improvement:

- Prediction accuracy should be defined using suitable evaluation metrics.
- Dashboard response time should be specified, for example, within 3 seconds under normal load.
- High-risk cases should be clearly defined using a specified probability threshold.

B. Redundancy

Some requirements may overlap with each other.

For example:

- "Monitor acquisition"
- "Monitor project progress"
- "View acquisition status"

These can be combined under a common Acquisition Monitoring feature while still providing different views for different users.

Improvement: Remove duplicate requirements and organize similar functions into common modules.

C. Inconsistency

Requirements should not conflict with each other.

For example, if the system allows the Revenue Officer to update approval information but does not allow the Land Acquisition Officer to view the updated information, there is an inconsistency.

Improvement:

- Define clear access permissions for each role.
- Ensure that updates made by one authorized department are reflected consistently throughout the system.
- Define a common source of truth for acquisition data.

D. Missing Requirements

The following requirements should be added:

1. Data Backup and Recovery – Regularly back up acquisition data.
2. Audit Logging – Record important user activities and changes.
3. Data Privacy – Protect landowner and project information.
4. Model Monitoring – Monitor prediction model performance.
5. Model Retraining – Retrain the model when new historical data becomes available.
6. Alert Acknowledgement – Allow officials to mark alerts as reviewed or resolved.
7. Data Validation – Prevent incorrect or incomplete data from being stored.

8. Disaster Recovery – Provide recovery mechanisms after system failures.

### 6.3 Suggested Improvements to Enhance Requirement Quality and Completeness

The following improvements can make the SRS more complete and effective:

- Define measurable performance requirements such as response time and availability.
- Clearly define risk categories such as Low, Medium, and High.
- Specify the prediction accuracy and evaluation criteria.
- Define role-based access permissions for every actor.
- Add backup and recovery requirements.
- Include data privacy and security requirements.
- Define how often the predictive model should be updated or retrained.
- Include audit logs for tracking changes.
- Define procedures for handling incorrect or missing data.
- Allow users to provide feedback on prediction results.
- Regularly review requirements with all stakeholders.
- Ensure that every functional requirement has a corresponding use case and test case.

Final Validation

The proposed SRS covers the major requirements of the Predictive Analytics System for Early Detection of Land Acquisition Delays. Most stakeholder needs are addressed through monitoring, predictive analysis, risk classification, alerts, and reporting. Further improvement can be achieved by defining measurable requirements, removing duplicate requirements, resolving possible inconsistencies, and adding security, backup, audit, and machine-learning model management requirements.